

Aplikasi Sistem Digital
Pintu otomatis dengan FSM (Finite State Machine)



Disusun Oleh:

Ahmad Hideo Alfarisi (1103223095)

Ariski Myardi (1103220019)

Gita Adi Pangestu (1103223225)

Khairan Abhipraya (1103223080)

Jurusan Teknik Komputer

Fakultas Teknik Elektro

Telkom University 2024

Latar Belakang

Sistem pintu otomatis telah menjadi solusi utama dalam berbagai aplikasi modern, seperti di pusat perbelanjaan, rumah sakit, dan gedung perkantoran. Menggunakan Finite State Machine (FSM) dalam desain sistem memastikan pengaturan yang terstruktur, responsif, dan aman. Penambahan sensor berat sebagai input memberikan lapisan tambahan untuk mendeteksi kehadiran objek atau orang dengan lebih akurat, sehingga meningkatkan efisiensi dan keamanan.

Spesifikasi Desain

1. Fungsi Utama

Sistem pintu otomatis dirancang untuk:

- Membuka dan menutup pintu berdasarkan input dari berbagai sensor (gerak, tombol manual, dan berat).
- Mencegah keadaan konflik seperti pintu membuka dan menutup secara bersamaan.
- Memiliki mekanisme reset untuk kondisi darurat.

2. Input Hardware

- Sensor Gerak: Mendeteksi keberadaan orang di sekitar pintu.
- Sensor Berat: Mengidentifikasi keberadaan atau beban seseorang/objek di dekat pintu, seperti kursi roda atau troli.
- Reset Button: Mengembalikan sistem ke kondisi awal dalam keadaan darurat.

3. Output Hardware

1. Output Motor Pintu (door_motor):

- **door_motor = 1: Pintu dalam keadaan terbuka.**
- **door_motor = 0: Pintu dalam keadaan tertutup.**

2. State LED (state_led):

- **state_led = "00": Pintu dalam keadaan tertutup.**
- **state_led = "00": Pintu dalam keadaan tertutup.**
- **state_led = "10" dan state_led = "11": Status lain yang dapat digunakan untuk menunjukkan keadaan antara terbuka atau tertutup, atau sebagai status tambahan sesuai dengan logika FSM yang lebih kompleks.**

3. State Management

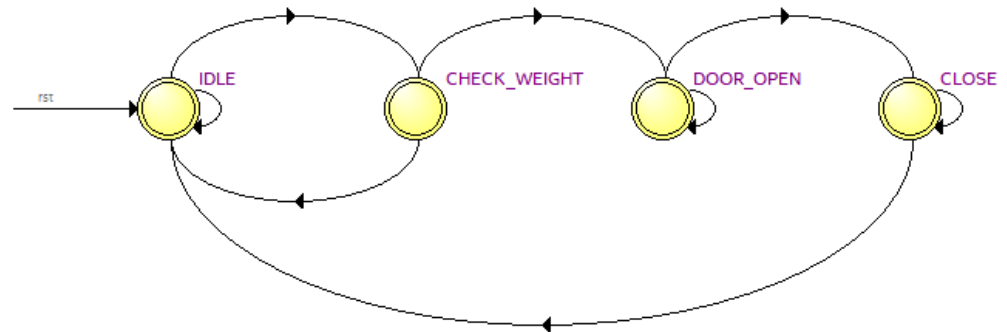
3.1 FSM untuk pintu ini terdiri dari beberapa keadaan yang masing-masing mewakili status pintu dan logika kontrol yang sesuai. Berikut adalah keadaan (states) yang digunakan dalam FSM:

- **State 0 (Pintu Tertutup - CLOSED):**
 - **Pintu dalam keadaan tertutup dan motor pintu dalam kondisi tidak aktif.**
 - **Transisi: Jika sensor gerakan mendeteksi gerakan dan sensor berat valid, FSM akan beralih ke keadaan terbuka.**
 - **Output: door_motor = 0 (Pintu tertutup), state_led = "00" (Indikator LED untuk pintu tertutup).**
- **State 1 (Pintu Terbuka - OPEN):**
 - **Pintu dalam keadaan terbuka, dan motor pintu dalam kondisi aktif.**
 - **Transisi: Jika tidak ada gerakan yang terdeteksi atau sensor berat tidak valid, FSM akan beralih kembali ke keadaan tertutup.**
 - **Output: door_motor = 1 (Pintu terbuka), state_led = "01" (Indikator LED untuk pintu terbuka).**

3.2 Proses transisi antar status dikendalikan oleh input yang diterima dari sensor-sensor dan juga oleh kondisi sistem saat ini. Setiap perubahan status terjadi pada setiap siklus clock, di mana FSM mengupdate status berdasarkan kondisi input.

- **Transisi dari State 0 (CLOSED) ke State 1 (OPEN):**
- Ketika sensor motion_detected mendeteksi gerakan dan sensor weight_valid menunjukkan berat yang valid, pintu akan dibuka, mengubah status ke OPEN (State 1).
- Transisi ini mengubah output door_motor menjadi 1 (motor aktif untuk membuka pintu) dan memperbarui state_led menjadi "01" untuk menunjukkan bahwa pintu dalam keadaan terbuka.
- **Transisi dari State 1 (OPEN) ke State 0 (CLOSED):**
 - Ketika tidak ada gerakan yang terdeteksi oleh sensor motion_detected atau sensor weight_valid menunjukkan berat yang tidak valid, pintu akan ditutup, mengubah status ke CLOSED (State 0).
 - Transisi ini mengubah output door_motor menjadi 0 (motor tidak aktif, pintu tertutup) dan memperbarui state_led menjadi "00" untuk menunjukkan pintu tertutup.

3.3 State Machine



4. Algoritma

1. Inisialisasi:

- Setel status pintu ke CLOSED (pintu tertutup).
- Setel motor pintu (door_motor) ke 0 (pintu tertutup).
- Setel indikator LED (state_led) ke "00" (menunjukkan pintu tertutup).

2. Proses:

- Jika ada sinyal clock dan tidak ada reset (reset = 0):
 - Jika sensor gerakan (motion_detected = 1) dan sensor berat (weight_valid = 1):
 - Pintu akan dibuka, jadi:
 - Status state: OPEN.
 - Output door_motor diatur menjadi 1 (motor aktif).

- Indikator LED (state_led) diatur menjadi "01".
- Jika tidak ada gerakan (motion_detected = 0) atau berat tidak valid (weight_valid = 0):
 - Pintu akan ditutup, jadi:
 - Status state: CLOSED.
 - Output door_motor diatur menjadi 0 (motor tidak aktif).
 - Indikator LED (state_led) diatur menjadi "00".

3. Reset :

- Ketika sinyal reset (rst = 1) diaktifkan, FSM mengatur status pintu kembali ke CLOSED, motor pintu (door_motor) menjadi 0, dan indikator LED (state_led) menjadi "00".

4. Selesai :

- Proses ini akan terus berulang berdasarkan sinyal clock, memeriksa input sensor dan mengubah status pintu.

KODE VHDL

```
library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

entity door_fsm is

    port (

        clk          : in  std_logic;          -- Clock signal

        rst          : in  std_logic;          -- Reset signal

        motion_detected : in  std_logic;        -- Input from motion sensor

        weight_valid   : in  std_logic;        -- Input from weight sensor (1 if valid weight)

        door_motor     : out std_logic;        -- Output to motor (1 = open, 0 = closed)

        state_led      : out std_logic_vector(1 downto 0) -- State indicator for debugging

    );

end door_fsm;

architecture Behavioral of door_fsm is

    type state_type is (CLOSED, OPEN); -- Define two states: CLOSED and OPEN

    signal state, next_state : state_type; -- Registers for the current and next state

begin

    -- Process for state transitions
```

```

process(clk, rst)
begin
    if rst = '1' then
        state <= CLOSED; -- Reset state to CLOSED

    elsif rising_edge(clk) then
        state <= next_state; -- Update state on the rising edge of the clock
    end if;
end process;

-- Logic for state transitions and output control
process(state, motion_detected, weight_valid)
begin
    -- Default values

    next_state <= state;

    door_motor <= '0'; -- Default motor state: door closed
    state_led <= "00"; -- Default LED state: door closed

    -- State transitions based on motion and weight
    case state is
        when CLOSED =>
            if motion_detected = '1' and weight_valid = '1' then
                next_state <= OPEN; -- Transition to OPEN state

                door_motor <= '1'; -- Open the door (motor on)
            end if;
        end case;
    end process;

```

```
        state_led <= "01"; -- Indicate OPEN state

    end if;

    when OPEN =>

        if motion_detected = '0' or weight_valid = '0' then

            next_state <= CLOSED; -- Transition to CLOSED state

            door_motor <= '0'; -- Close the door (motor off)

            state_led <= "00"; -- Indicate CLOSED state

        end if;

    end case;

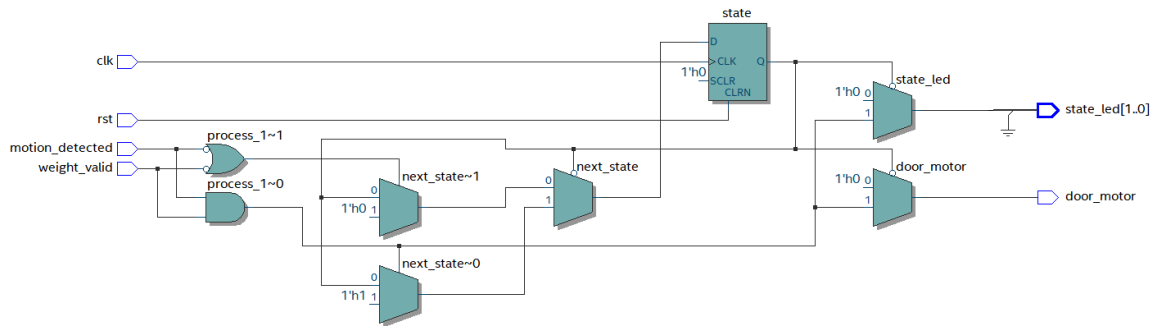
end process;

end Behavioral;
```

DESAIN FSM

Keadaan (State)	Input	Transisi Keadaan	Output
CLOSED	motion_detected = 1 dan weight_valid = 1	Pintu terbuka (OPEN)	door_motor = 1 (pintu terbuka), state_led = "01"
CLOSED	motion_detected = 0 atau weight_valid = 0	Tetap di CLOSED	door_motor = 0 (pintu tertutup), state_led = "00"
OPEN	motion_detected = 0 atau weight_valid = 0	Pintu tertutup (CLOSED)	door_motor = 0 (pintu tertutup), state_led = "00"
OPEN	motion_detected = 1 dan weight_valid = 1	Tetap di OPEN	door_motor = 1 (pintu terbuka), state_led = "01"

Sequential FSM Pintu Otomatis:



KODE :

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

-- Entity declaration

entity door_fsm_seq is

port (

clk : in std_logic; -- Clock signal

rst : in std_logic; -- Reset signal

motion_detected : in std_logic; -- Input from motion sensor

weight_valid : in std_logic; -- Input from weight sensor (1 if valid weight)

door_motor : out std_logic; -- Output to motor (1 = open, 0 = closed)

state_led : out std_logic_vector(1 downto 0) -- State indicator for debugging

```

);

end door_fsm_seq;

-- Architecture definition

architecture Behavioral of door_fsm_seq is

    -- Define the state type (CLOSED and DOOR_OPEN are the two states)

    type state_type is (CLOSED, DOOR_OPEN);

    signal state, next_state : state_type; -- Registers for the current and next state

begin

    -- Sequential process for state transitions (based on clock and reset)

    process(clk, rst)

    begin

        if rst = '1' then

            state <= CLOSED; -- Reset state to CLOSED

        elsif rising_edge(clk) then

            state <= next_state; -- Update state on the rising edge of the clock

        end if;

    end process;

    -- Logic for state transitions and output control (combinational)

    process(state, motion_detected, weight_valid)

    begin

        -- Default values (hold state until condition changes)

```

```

next_state <= state; -- Stay in current state if no transition occurs

door_motor <= '0';    -- Default motor state: door closed

state_led <= "00";    -- Default LED state: door closed


-- State transitions based on motion and weight sensor inputs

case state is

    when CLOSED =>

        -- If motion detected and weight is valid, open the door

        if motion_detected = '1' and weight_valid = '1' then

            next_state <= DOOR_OPEN; -- Transition to DOOR_OPEN state

            door_motor <= '1';        -- Open the door (motor on)

            state_led <= "01";        -- Indicate DOOR_OPEN state

        end if;

    when DOOR_OPEN =>

        -- If no motion or weight is invalid, close the door

        if motion_detected = '0' or weight_valid = '0' then

            next_state <= CLOSED; -- Transition to CLOSED state

            door_motor <= '0';        -- Close the door (motor off)

            state_led <= "00";        -- Indicate CLOSED state

        end if;

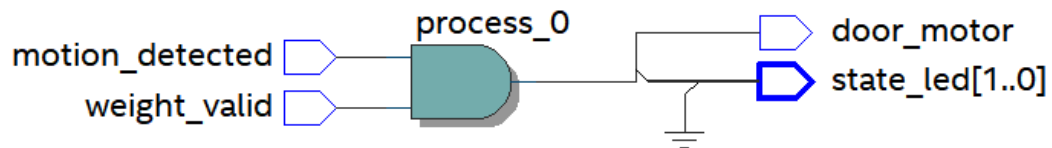
end case;

```

end process;

end Behavioral;

KOMBINANTIONAL FSM PINTU OTOMATIS :



KODE :

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

-- Entity declaration

entity door_fsm_comb is

port (

motion_detected : in std_logic; -- Input dari sensor gerakan

weight_valid : in std_logic; -- Input dari sensor berat (1 jika berat valid)

door_motor : out std_logic; -- Output untuk motor pintu (1 = buka, 0 = tutup)

```

        state_led      : out std_logic_vector(1 downto 0) -- Output untuk indikator LED (status
pintu)

    );
end door_fsm_comb;

```

-- Architecture definition

architecture Behavioral of door_fsm_comb is

-- Definisikan tipe data untuk status (CLOSED dan DOOR_OPEN)

type state_type is (CLOSED, DOOR_OPEN);

signal state, next_state : state_type; -- Sinyal untuk status saat ini dan status berikutnya

begin

-- Kombinatorial process untuk transisi status dan output kontrol motor pintu

process(motion_detected, weight_valid, state)

begin

-- Default values untuk output

next_state <= state; -- Tetap di status saat ini jika tidak ada transisi

door_motor <= '0'; -- Default motor: pintu tertutup

state_led <= "00"; -- Default LED: pintu tertutup (CLOSED)

-- Transisi status berdasarkan input motion dan weight

case state is

when CLOSED =>

-- Jika gerakan terdeteksi dan berat valid, buka pintu

```

if motion_detected = '1' and weight_valid = '1' then

    next_state <= DOOR_OPEN; -- Transisi ke status DOOR_OPEN

    door_motor <= '1';    -- Motor aktif, pintu terbuka

    state_led <= "01";    -- Indikator LED DOOR_OPEN

end if;


when DOOR_OPEN =>

    -- Jika tidak ada gerakan atau berat tidak valid, tutup pintu

    if motion_detected = '0' or weight_valid = '0' then

        next_state <= CLOSED; -- Transisi ke status CLOSED

        door_motor <= '0';    -- Motor nonaktif, pintu tertutup

        state_led <= "00";    -- Indikator LED CLOSED

    end if;


end case;

end process;


end Behavioral;

```

Tantangan

Level 1 : Integrasi Sensor yang Beragam

- Menghubungkan sensor gerak dan sensor berat dengan sistem FSM.
- Memastikan input sensor dapat diterjemahkan menjadi sinyal digital yang stabil.

Level 2 : Keamanan dan Keandalan

- Mencegah false positive dari sensor yang menyebabkan pintu terbuka tanpa alasan.
- Menjamin sistem tetap berfungsi meskipun salah satu sensor mengalami gangguan.

Level 3 : Pengujian dan Validasi Sistem yang Kompleks

- Melakukan simulasi kondisi ekstrem, seperti input sensor yang bervariasi secara cepat atau skenario darurat kompleks.
- Menyusun protokol uji coba yang mencakup semua kemungkinan kasus.

Kesimpulan

Sistem pintu otomatis berbasis Finite State Machine (FSM) merupakan solusi efektif untuk menciptakan pengendalian pintu yang terstruktur, aman, dan responsif. Implementasi FSM memungkinkan sistem untuk beradaptasi dengan berbagai input sensor seperti sensor gerak dan berat, sehingga memastikan efisiensi dalam operasional.

Melalui desain FSM yang sederhana namun tangguh, transisi antarstatus pintu dapat dilakukan dengan akurat sesuai kondisi lingkungan. Fitur tambahan seperti mekanisme reset memberikan kemampuan penanganan keadaan darurat yang andal. Proses pengujian sistem, termasuk simulasi kondisi ekstrem, menunjukkan bahwa sistem ini dapat diandalkan dalam berbagai aplikasi nyata, seperti di pusat perbelanjaan, rumah sakit, dan gedung perkantoran.

Penggunaan FSM dalam sistem pintu otomatis tidak hanya meningkatkan kenyamanan tetapi juga menambah lapisan keamanan bagi pengguna. Dengan begitu, desain ini menjadi langkah penting menuju pengembangan teknologi berbasis otomasi yang lebih cerdas dan adaptif.